Optimization and Stochastic Hill Climbing

Table of Contents

Introduction

Optimization is finding the best solution from a set of possible solutions. In the context of maximum likelihood estimation, we want to find parameters that maximize the log-likelihood.

Why we need optimization:

- Analytical solutions don't always exist
- Complex models require numerical methods
- Can handle constraints on parameters

Types of optimization:

1. **Gradient-based**: Use derivatives (fast, but need gradients)
2. **Gradient-free**: Don't need derivatives (slower, more robust)

- **Stochastic hill climbing** (what we'll use!)
- Simulated annealing
- Genetic algorithms

Optimization Basics

The Optimization Problem

General form:

```
θ* = argmin_θ f(θ)    [minimization]
θ* = argmax_θ f(θ)    [maximization]
```

For MLE:

```
θ* = argmax_θ ℓ(θ)         [maximize log-likelihood]
θ* = argmin_θ [-ℓ(θ)]      [minimize negative log-likelihood]
```

Objective Function

The function we're optimizing is called the **objective function**:

```
def objective(theta, observations):
    """
    Objective function to minimize

    For MLE, this is negative log-likelihood
    """
    # Compute log-likelihood
    llik = compute_log_likelihood(theta, observations)

    # Return negative (for minimization)
    return -llik
```

Constraints

Often parameters have **constraints**:

- Probabilities: $0 \leq p \leq 1$
- Variance: $\sigma^2 > 0$
- Coordinates: $0 \leq x \leq 16$
- Angles: $0° \leq \text{angle} \leq 45°$

Handling constraints:

1. **Reject**: Return infinity for invalid parameters
2. **Project**: Clip parameters to valid range
3. **Transform**: Use transformed variables (e.g., $\log(\sigma)$ instead of σ)
4. **Penalty**: Add penalty term to objective

Hill Climbing Algorithm

Basic Idea

Hill climbing is like climbing a mountain in fog:

1. Start somewhere
2. Look around nearby
3. Move to the highest point you can see
4. Repeat until you can't go higher

Deterministic Hill Climbing

Algorithm:

```
1. Start with initial guess θ₀
2. Evaluate f(θ₀)
3. Generate neighbors of θ₀
4. Evaluate f for each neighbor
5. Move to best neighbor if it's better
6. Repeat until no improvement
```

Problem: Can get stuck in local maxima!

```
def hill_climbing(objective, theta_init, step_size=0.1, max_iter=1000):
    """
    Simple hill climbing

    Parameters:
    -----------
    objective : callable
        Function to minimize f(theta)
    theta_init : array
        Initial parameters
    step_size : float
        Size of steps to take
    max_iter : int
        Maximum iterations

    Returns:
    --------
    theta_best : array
        Best parameters found
    """
    theta = theta_init.copy()
    f_current = objective(theta)

    for iteration in range(max_iter):
        improved = False

        # Try small changes in each dimension
        for i in range(len(theta)):
            # Try increasing
            theta_new = theta.copy()
            theta_new[i] += step_size
            f_new = objective(theta_new)

            if f_new < f_current:  # Better!
                theta = theta_new
                f_current = f_new
                improved = True
                continue

            # Try decreasing
```

```
            theta_new = theta.copy()
            theta_new[i] -= step_size
            f_new = objective(theta_new)

            if f_new < f_current:  # Better!
                theta = theta_new
                f_current = f_new
                improved = True

        # If no improvement, we're at a local optimum
        if not improved:
            break

    return theta
```

Stochastic Hill Climbing

Why Stochastic?

Problems with deterministic hill climbing:

- Gets stuck in local optima
- Sensitive to initial conditions
- Rigid search pattern

Solution: Add randomness!

- Random perturbations
- Random acceptance
- Can escape local optima

Algorithm

Stochastic hill climbing:

```
1. Start with random θ
2. Repeat:
   a. Generate random perturbation Δθ
   b. Evaluate θ_new = θ + Δθ
   c. If f(θ_new) < f(θ): accept (move to θ_new)
   d. Otherwise: reject (stay at θ)
3. Return best θ found
```

Key Components

1. Random Perturbation

```
def perturb(theta, step_size):
    """Generate random perturbation"""
    delta = np.random.randn(len(theta)) * step_size
    return theta + delta
```

2. Acceptance Criterion

```
def accept(f_new, f_current):
    """Simple greedy acceptance: only accept improvements"""
    return f_new < f_current
```

3. Step Size Schedule

```
def get_step_size(iteration, initial_step=0.1, decay=0.99):
    """Decrease step size over time"""
    return initial_step * (decay ** iteration)
```

Implementation

```
def stochastic_hill_climbing(objective, theta_init, step_size=0.1,
                             max_iter=1000, patience=100):
    """
    Stochastic hill climbing optimizer

    Parameters:
    -----------
    objective : callable
        Function to minimize f(theta)
    theta_init : array
        Initial parameters
    step_size : float or callable
        Step size (or function that returns step size given iteration)
    max_iter : int
        Maximum iterations
    patience : int
        Stop if no improvement for this many iterations

    Returns:
    --------
    theta_best : array
        Best parameters found
    f_best : float
        Best objective value
    history : dict
        Optimization history
    """
    theta_current = theta_init.copy()
    f_current = objective(theta_current)

    theta_best = theta_current.copy()
    f_best = f_current

    history = {
        'theta': [theta_current.copy()],
        'f': [f_current],
        'accepted': []
    }
```

```
    no_improvement = 0

    for iteration in range(max_iter):
        # Get step size (can decrease over time)
        if callable(step_size):
            current_step = step_size(iteration)
        else:
            current_step = step_size

        # Generate random perturbation
        delta = np.random.randn(len(theta_current)) * current_step
        theta_new = theta_current + delta

        # Evaluate
        f_new = objective(theta_new)

        # Accept if better
        if f_new < f_current:
            theta_current = theta_new
            f_current = f_new
            accepted = True

            # Update best
            if f_new < f_best:
                theta_best = theta_new.copy()
                f_best = f_new
                no_improvement = 0
            else:
                no_improvement += 1
        else:
            accepted = False
            no_improvement += 1

        # Record history
        history['theta'].append(theta_current.copy())
        history['f'].append(f_current)
        history['accepted'].append(accepted)

        # Early stopping
        if no_improvement >= patience:
```

```
            break

    return theta_best, f_best, history
```

Handling Constraints

Method 1: Rejection

Simply reject invalid parameters:

```
def objective_with_rejection(theta, observations, bounds):
    """
    Objective function that rejects invalid parameters

    Parameters:
    -----------
    theta : array
        Parameters
    observations : array
        Data
    bounds : list of tuples
        [(min_0, max_0), (min_1, max_1), ...]
    """
    # Check bounds
    for i, (low, high) in enumerate(bounds):
        if theta[i] < low or theta[i] > high:
            return np.inf  # Invalid! Return worst possible value

    # Valid parameters: compute objective
    return compute_objective(theta, observations)
```

Method 2: Projection (Clipping)

Project invalid parameters back to valid range:

```
def project_to_bounds(theta, bounds):
    """
    Project parameters to satisfy bounds

    Parameters:
    -----------
    theta : array
        Parameters (possibly invalid)
    bounds : list of tuples
        [(min_0, max_0), (min_1, max_1), ...]

    Returns:
    --------
    theta_valid : array
        Parameters clipped to valid range
    """
    theta_valid = theta.copy()
    for i, (low, high) in enumerate(bounds):
        theta_valid[i] = np.clip(theta_valid[i], low, high)
    return theta_valid
```

```
def stochastic_hill_climbing_with_projection(objective, theta_init,
                                             bounds, step_size=0.1,
                                             max_iter=1000):
    """Stochastic hill climbing with parameter projection"""

    theta_current = project_to_bounds(theta_init, bounds)
    f_current = objective(theta_current)

    theta_best = theta_current.copy()
    f_best = f_current

    for iteration in range(max_iter):
        # Perturb
        delta = np.random.randn(len(theta_current)) * step_size
        theta_new = theta_current + delta

        # Project to valid range
        theta_new = project_to_bounds(theta_new, bounds)

        # Evaluate
        f_new = objective(theta_new)

        # Accept if better
        if f_new < f_current:
            theta_current = theta_new
            f_current = f_new

            if f_new < f_best:
                theta_best = theta_new.copy()
                f_best = f_new

    return theta_best, f_best
```

Method 3: Adaptive Step Size

Adjust step size based on success rate:

```
def adaptive_stochastic_hill_climbing(objective, theta_init,
                                      initial_step=0.1, max_iter=1000):
    """Stochastic hill climbing with adaptive step size"""

    theta_current = theta_init.copy()
    f_current = objective(theta_current)
    step_size = initial_step

    theta_best = theta_current.copy()
    f_best = f_current

    recent_accepts = []

    for iteration in range(max_iter):
        # Perturb
        delta = np.random.randn(len(theta_current)) * step_size
        theta_new = theta_current + delta

        # Evaluate
        f_new = objective(theta_new)

        # Accept if better
        accepted = (f_new < f_current)
        if accepted:
            theta_current = theta_new
            f_current = f_new

            if f_new < f_best:
                theta_best = theta_new.copy()
                f_best = f_new

        # Track recent acceptance rate
        recent_accepts.append(accepted)
        if len(recent_accepts) > 100:
            recent_accepts.pop(0)

        # Adjust step size
        if len(recent_accepts) >= 50:
            acceptance_rate = np.mean(recent_accepts)
```

```
            if acceptance_rate > 0.5:  # Too many accepts → increase step
                step_size *= 1.1
            elif acceptance_rate < 0.2:  # Too few accepts → decrease step
                step_size *= 0.9

    return theta_best, f_best
```

MLE Implementation for Cross Distribution

Problem Setup

Goal: Find parameters of CrossDistribution that maximize likelihood of observations.

Parameters:

- `ctr` : [ctr_x, ctr_y] ∈ [0, 16] × [0, 16]
- `span` : ∈ [0, 20]
- `leg` : ∈ [0, 10]
- `angle` : ∈ [0, 45]

Total: 5 parameters

Objective Function

```
def negative_log_likelihood_cross(theta, observations, CrossDistribution):
    """
    Negative log-likelihood for cross distribution

    Parameters:
    -----------
    theta : array (5,)
        [ctr_x, ctr_y, span, leg, angle]
    observations : array (n, 2)
        Observed submarine locations
    CrossDistribution : class
        Cross distribution class

    Returns:
    --------
    neg_llik : float
        Negative log-likelihood (or inf if invalid)
    """
    # Extract parameters
    ctr_x, ctr_y, span, leg, angle = theta

    # Check constraints
    if not (0 <= ctr_x <= 16 and 0 <= ctr_y <= 16):
        return np.inf
    if not (0 <= span <= 20):
        return np.inf
    if not (0 <= leg <= 10):
        return np.inf
    if not (0 <= angle <= 45):
        return np.inf

    # Create distribution
    try:
        dist = CrossDistribution(ctr=(ctr_x, ctr_y), span=span,
                                 leg=leg, angle=angle)
    except:
        return np.inf  # Invalid parameters
```

```
    # Compute log-likelihood
    log_likelihood = 0.0
    for obs in observations:
        llik = dist.llik(obs)
        if np.isfinite(llik):
            log_likelihood += llik
        else:
            return np.inf  # Invalid

    # Return negative
    return -log_likelihood
```

Optimization

```
def optimize_cross_distribution(observations, CrossDistribution,
                                n_restarts=10, max_iter=15000):
    """
    Find MLE parameters for cross distribution

    Parameters:
    -----------
    observations : array (n, 2)
        Observed data
    CrossDistribution : class
        Distribution class
    n_restarts : int
        Number of random restarts
    max_iter : int
        Maximum iterations per restart

    Returns:
    --------
    theta_best : array (5,)
        Best parameters found
    """
    # Bounds
    bounds = [
        (0, 16),   # ctr_x
        (0, 16),   # ctr_y
        (0, 20),   # span
        (0, 10),   # leg
        (0, 45),   # angle
    ]

    best_theta = None
    best_f = np.inf

    for restart in range(n_restarts):
        # Random initialization
        theta_init = np.array([
            np.random.uniform(0, 16),  # ctr_x
            np.random.uniform(0, 16),  # ctr_y
```

```
            np.random.uniform(0, 20),  # span
            np.random.uniform(0, 10),  # leg
            np.random.uniform(0, 45),  # angle
        ])

        # Optimize
        theta, f = stochastic_hill_climbing_with_projection(
            lambda theta: negative_log_likelihood_cross(
                theta, observations, CrossDistribution
            ),
            theta_init=theta_init,
            bounds=bounds,
            step_size=0.1,
            max_iter=max_iter
        )

        # Update best
        if f < best_f:
            best_theta = theta
            best_f = f

    return best_theta
```

Tuning and Debugging

Hyperparameters to Tune

1. Step size

- Too large: jumps around, doesn't converge
- Too small: slow convergence, gets stuck
- **Tip**: Start with ~0.1 and adjust

2. Maximum iterations

- Too few: doesn't converge

- Too many: wastes time
- **Tip**: Monitor convergence, stop when no improvement

3. Number of restarts

- More restarts = better global search
- **Tip**: Use 5-10 restarts for robust results

Monitoring Convergence

```
def plot_convergence(history):
    """Plot optimization history"""
    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(history['f'])
    plt.xlabel('Iteration')
    plt.ylabel('Objective Value')
    plt.title('Convergence')
    plt.yscale('log')

    plt.subplot(1, 2, 2)
    accepts = np.array(history['accepted'])
    window = 100
    acceptance_rate = np.convolve(accepts.astype(float),
                                  np.ones(window)/window,
                                  mode='valid')
    plt.plot(acceptance_rate)
    plt.xlabel('Iteration')
    plt.ylabel('Acceptance Rate (rolling)')
    plt.title('Acceptance Rate')

    plt.tight_layout()
    plt.show()
```

Common Issues

Issue 1: Not converging

- **Cause**: Step size too large or too small
- **Fix**: Adjust step size, use adaptive step size

Issue 2: Stuck in local optimum

- **Cause**: Bad initialization
- **Fix**: Use multiple random restarts

Issue 3: Parameters hit bounds repeatedly

- **Cause**: Bounds too tight or wrong initialization
- **Fix**: Check bounds, use better initialization

Issue 4: Objective is inf or nan

- **Cause**: Invalid parameters or numerical issues
- **Fix**: Add parameter validation, use log-space

Complete Example

```
import numpy as np
from submarine import submarine_observations, CrossDistribution

def objective(theta, observations):
    """Negative log-likelihood"""
    ctr_x, ctr_y, span, leg, angle = theta

    # Enforce constraints
    if not (0 <= ctr_x <= 16): return np.inf
    if not (0 <= ctr_y <= 16): return np.inf
    if not (0 <= span <= 20): return np.inf
    if not (0 <= leg <= 10): return np.inf
    if not (0 <= angle <= 45): return np.inf

    # Create model
    model = CrossDistribution(ctr=(ctr_x, ctr_y), span=span,
                              leg=leg, angle=angle)

    # Compute negative log-likelihood
    llik = sum([model.llik(obs) for obs in observations])
    return -llik if np.isfinite(llik) else np.inf

def optimize(observations, max_iter=15000):
    """Find best parameters"""

    # Multiple random restarts
    best_theta = None
    best_f = np.inf

    for _ in range(10):
        # Random initialization
        theta_init = np.array([
            np.random.uniform(4, 12),   # ctr_x
            np.random.uniform(4, 12),   # ctr_y
            np.random.uniform(0.1, 5),  # span
            np.random.uniform(1, 8),    # leg
            np.random.uniform(10, 40),  # angle
```

```
        ])

        # Optimize
        theta = theta_init.copy()
        f = objective(theta, observations)

        step_size = 0.1

        for iteration in range(max_iter):
            # Adaptive step size
            if iteration % 1000 == 0:
                step_size *= 0.95

            # Random perturbation
            delta = np.random.randn(5) * step_size
            theta_new = theta + delta

            # Clip to bounds
            theta_new[0] = np.clip(theta_new[0], 0, 16)
            theta_new[1] = np.clip(theta_new[1], 0, 16)
            theta_new[2] = np.clip(theta_new[2], 0, 20)
            theta_new[3] = np.clip(theta_new[3], 0, 10)
            theta_new[4] = np.clip(theta_new[4], 0, 45)

            # Evaluate
            f_new = objective(theta_new, observations)

            # Accept if better
            if f_new < f:
                theta = theta_new
                f = f_new

        # Update best
        if f < best_f:
            best_theta = theta
            best_f = f

    return best_theta

# Run optimization
np.random.seed(2025)
```

```
theta_opt = optimize(submarine_observations, max_iter=15000)

print("Optimal parameters:")
print(f"  ctr: ({theta_opt[0]:.2f}, {theta_opt[1]:.2f})")
print(f"  span: {theta_opt[2]:.2f}")
print(f"  leg: {theta_opt[3]:.2f}")
print(f"  angle: {theta_opt[4]:.2f}")

# Visualize fit
model_opt = CrossDistribution(ctr=(theta_opt[0], theta_opt[1]),
                              span=theta_opt[2], leg=theta_opt[3],
                              angle=theta_opt[4])
model_opt.draw()
plt.scatter(submarine_observations[:, 0],
            submarine_observations[:, 1],
            c='white', s=10, label='Observations')
plt.legend()
plt.title('Optimized Fit')
plt.show()
```

Summary

Stochastic Hill Climbing:

```
1. Initialize randomly
2. Perturb randomly
3. Accept if better
4. Repeat
5. Return best found
```

Key Components:

- **Objective function**: What to minimize/maximize
- **Perturbation**: Random changes to parameters
- **Acceptance**: Greedy (only accept improvements)

- **Step size**: Controls exploration vs exploitation
- **Constraints**: Bounds on parameters

Best Practices:

1. Use multiple random restarts
2. Monitor convergence (plot objective over iterations)
3. Adjust step size (decrease over time)
4. Enforce constraints (reject or project)
5. Use log-likelihood for numerical stability
6. Limit iterations (15,000 max for lab)

For MLE:

- Objective = negative log-likelihood
- Parameters = distribution parameters
- Constraints = valid parameter ranges
- Goal = find parameters that best explain observed data

Advantages:

- Simple to implement
- No derivatives needed
- Can handle constraints easily
- Can escape some local optima (with multiple restarts)

Disadvantages:

- Slower than gradient-based methods
- No convergence guarantees
- Requires tuning hyperparameters
- May need many iterations

Improvements:

- Simulated annealing (accept worse solutions sometimes)
- Adaptive step sizes
- Multiple restarts

- Hybrid methods (combine with gradient descent)